Subsystem Overview

SyncSpace is decomposed into **five main subsystems**. Each subsystem has clear boundaries and communicates with others through well-defined interfaces.

---

1. Editor Frontend Subsystem

Role: Provides the user-facing code editing interface and renders all visual collaboration elements.

Functionality:

- Hosts the code editor (**CodeMirror 6**) with syntax highlighting (user-selectable **Python, C++, Java**), line numbers, and standard editing affordances
- Renders remote users' cursors and selections as colored overlays
- Displays the participant list, connection status, optional **display name** editing, and a toggle for **cursor name** visibility
- Provides UI controls for sessions: **new session** via HTTP redirect from `/` to `/app/?session=…` (short id issued by the server), **join** via the same query parameter on a shared link; **Share link** fetches `/api/share-link` so the clipboard gets a **LAN-routable or public base URL** when configured, not only `127.0.0.1`
- Captures local edit operations and forwards them to the CRDT Engine

Key Technologies: HTML/CSS/JavaScript, **CodeMirror 6**, ES modules + import maps (CDN-hosted editor/CRDT packages)

Interfaces:

- → CRDT Engine: passes local edit operations, receives remote operations for rendering
- → Awareness Module: sends local cursor position, receives remote cursor data
- → WebSocket Client: indirectly via the CRDT engine's network provider

---

2. CRDT Engine Subsystem

Role: Manages the shared document state using Conflict-free Replicated Data Types to guarantee eventual consistency across all connected clients.

Functionality:

- Maintains a local Yjs `Y.Doc` replica of the shared document
- Applies local edits instantly to the replica (local-first, no network wait)
- Merges incoming remote edits automatically using CRDT merge semantics
- Guarantees that all replicas converge to an identical state regardless of edit order or timing
- Encodes incremental updates as compact binary state vectors for efficient network transfer

Key Technologies: Yjs (CRDT library)

Interfaces:

- ← Editor Frontend: receives local edit events
- → Editor Frontend: emits remote update events for re-rendering
- ↔ Communication Subsystem: sends/receives CRDT update vectors over the WebSocket channel

Architectural Significance: This is the core subsystem that fulfills FR-CE-01 (local-first editing), FR-CE-03 (conflict resolution), and NFR-R-01 (100% convergence guarantee). The choice of CRDT over Operational

Transformation is driven by this subsystem.

---

3. Communication Subsystem (WebSocket Layer)

Role: Provides the real-time bidirectional transport layer between clients and the relay server.

Functionality:

- Establishes and maintains a persistent WebSocket connection (`ws://` **in development**; **WSS** in production) between each client and the relay server
- Transmits **prefixed** binary frames: **document** updates (`0x00` + Yjs update) and **awareness** updates (`0x01` + awareness payload); optional **JSON** control messages (e.g. `request_state`) for peer-driven catch-up
- Receives the same from peers **via the relay** and delivers them to the CRDT Engine / Awareness layer
- **Prototype:** on unexpected disconnect, the client uses a **simple timed page reload** rather than a full exponential-backoff policy

Wire Protocol Summary:

Frame type	Format	Direction	Purpose
Document update	Binary: `0x00` + Yjs update bytes	Client ↔ Relay	CRDT state synchronization
Awareness update	Binary: `0x01` + awareness payload bytes	Client ↔ Relay	Cursor/presence broadcasting
State request	JSON text: `{"type": "request_state"}`	Joiner → Relay → Peers	New/reconnecting client requests full state from existing peers

Limitation — Destructive Page Reload: A page reload destroys the in-memory `Y.Doc` on that client. Any local edits not yet synced to at least one peer are permanently lost. A production system should reconnect the WebSocket while preserving the local `Y.Doc` instance rather than reloading the page.

Key Technologies: Browser **WebSocket API**; **custom** `SimpleProvider` in the client; server uses **FastAPI** WebSockets

Interfaces:

- ↔ CRDT Engine: transports CRDT update vectors
- ↔ Awareness Module: transports cursor/presence data
- ↔ Relay Server: the network endpoint

Architectural Significance: This subsystem directly addresses NFR-P-01 (≤ 1s E2E latency under normal conditions) by using a **full-duplex** WebSocket instead of HTTP polling. **Catch-up after join or disconnect** is handled by **Yjs state exchange between peers** (full update via `encodeStateAsUpdate` / `applyUpdate`), not by interpreting updates on the relay.

---

4. Relay Server Subsystem

Role: Acts as a lightweight, stateless message router that forwards CRDT updates and awareness data between connected clients within a session.

Functionality:

- Serves the static client under `/app` and redirects `/` to a new session URL
- Exposes `GET /api/share-link?session=…` to build a copy-pasteable URL (optional env: `SYNCSPACE_PUBLIC_BASE`, `SYNCSPACE_PUBLIC_HOST`, `SYNCSPACE_PORT`)
- Accepts WebSocket connections at `/ws/{session_id}` and groups peers by **session ID**
- Forwards (**broadcasts**) each received **text or binary** frame to **all other** sockets in that session **unchanged**
- Does **not** parse Yjs or awareness payloads; does **not** persist the document
- Maintains **only** an in-memory **set of active WebSockets per session**; when the last peer leaves, the session entry is removed

Key Technologies: Python 3, **FastAPI**, **Uvicorn** (ASGI server)

Interfaces:

- ↔ Communication Subsystem (each client): WebSocket connections
- Internally: session registry (in-memory map of session ID → connected clients)

Architectural Significance: The relay is intentionally **stateless** in terms of edit logic — it does not interpret, transform, or resolve edits. All conflict resolution happens at the CRDT layer on each client. This design avoids a central bottleneck and keeps the server simple.

Limitation — Session Loss: Because the relay holds no document state, if all clients disconnect simultaneously the session's document is irrecoverable from the relay alone. This is an accepted prototype limitation (see ADR-007). Recovery depends entirely on at least one client retaining its local Y.Doc replica.

---

5. Awareness / Presence Subsystem

Role: Tracks and broadcasts ephemeral collaboration state — specifically, each user's cursor position, text selection, display name, and online status.

Functionality:

- Each client publishes its local cursor position and selection range whenever they change
- The subsystem broadcasts this data to all peers via the relay (using Yjs Awareness protocol)
- Remote awareness states are rendered as colored cursor markers and selection highlights in the editor
- Detects user timeout/disconnection and removes stale awareness entries

Key Technologies: Yjs Awareness (`y-protocols/awareness`), transported over the same WebSocket as document updates with a **distinct prefix**

Interfaces:

- → Editor Frontend: provides remote cursor/selection data for rendering
- ← Editor Frontend: receives local cursor/selection changes
- ↔ Communication Subsystem: awareness data travels on the same WebSocket but as a logically separate channel

Architectural Significance: This subsystem is architecturally notable because it requires a **separate data model** from the document CRDT. Awareness data is ephemeral (not persisted, not conflict-resolved) and uses a distinct protocol from the document updates, justifying its separation as an independent subsystem.

Subsystem Interaction Diagram

```
┌──────────────────────────────────────────────────┐
│                     CLIENT                       │
│                                                  │
│   ┌──────────────┐    edits    ┌──────────────┐   │
│   │   Editor     │◄──────────► │  CRDT Engine │   │
│   │  Frontend    │   updates   │   (Yjs)      │   │
│   └──────────────┘             └──────────────┘   │
│          │ cursor                     │ CRDT vectors │
│          ▼                            ▼           │
│   ┌──────────────┐             ┌──────────────┐   │
│   │  Awareness   │◄──────────► │  WebSocket   │   │
│   │  Module      │  awareness  │  Client      │   │
│   └──────────────┘    data     └──────────────┘   │
│                                        │          │
└────────────────────────────────────────┼──────────┘
                                         │ WebSocket (ws / wss)
                                         ▼
                              ┌──────────────────┐
                              │  Relay Server    │
                              │  (FastAPI)       │
                              │  - broadcast     │
                              │  - session mgmt  │
                              │  - /app static   │
                              └──────────────────┘
                                         │ WebSocket (ws / wss)
                                         ├──────────────────────┐
                                         ▼                      ▼
                              ┌──────────────┐         ┌──────────────┐
                              │ Client B     │         │ Client C     │
                              └──────────────┘         └──────────────┘
```